Run the 7-step process on a mini scenario — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `seven_step_practice.md`: the 7-step planning process only actually becomes a habit once you've run it on a real, messy request, not just read about it. This one small scenario is that first real run, done on paper with no code at stake, before any actual project depends on getting it right. **If not:** the first time you ever tried the process for real would be at the very start of Project 1, with a whole project's worth of decisions riding on getting the process itself right.

Write your own answer in `practice/seven_step_practice.md` first, then compare.

Scenario: *"Build something that reads a team's daily emails and flags the urgent ones."*

Solution

Story: a plan that says "an urgency-checker" needs more meetings before anyone can actually build it; one that says `judge_urgency(email) -> (is_urgent, reason)` can be started right now, by anyone. Naming real function signatures and concrete data shapes is what turns a plan from an idea into work you can genuinely start today. **If not:** every vague step in the plan would turn into a design argument halfway through coding, after time has already been spent.

1. **Problem:** a team's shared inbox receives more email than anyone reads carefully; genuinely urgent messages (a client escalation, a deadline today) sometimes sit unread alongside routine ones. We need a system that reviews new emails once a day and surfaces the ones that need immediate attention, without anyone having to read every message.

2. **Requirements:** - Must: read every new email since the last run, judge each one's urgency, write the urgent ones to a place the team actually checks, include *why* each one was flagged. - Must never: modify or delete the original email, silently drop an email without judging it, flag something as urgent with no visible reason.

3. **Architecture:** a single agent, not multiple — this is a classification task (urgent or not) applied to each email independently, with no multi-step reasoning or handoff needed between different specialists. Tying this to Doc11: multiple agents earn their cost when a task genuinely needs different specialized skills working together; a single well-prompted classifier does this job.

4. **Components:** `fetch_new_emails()` (reads from the inbox since last run), `judge_urgency(email) -> (is_urgent, reason)` (the actual agent call), `write_flagged(email, reason)` (writes to wherever the team checks — a shared doc, a Slack message, a labeled inbox folder).

5. **Data Flow:** `fetch_new_emails()` returns a list → each email passed one at a time to `judge_urgency()` → for each `True` result, `write_flagged()` records the email and the reason → nothing happens for `False` results.

6. **Implementation Plan:** first, get `judge_urgency()` working and tested against 5-10 hand-picked example emails (some obviously urgent, some obviously not, a couple of genuinely ambiguous ones) — this is the actual hard part, and it should work correctly before anything touches a real inbox. Then wire up `fetch_new_emails()` against the real email source. Then wire up `write_flagged()`.

7. **Coding Tasks:** (a) write `judge_urgency()` with a clear prompt and a Pydantic-shaped return (`is_urgent: bool`, `reason: str`), (b) write and run a test file with 8-10 example emails and manually check the results, (c) write `fetch_new_emails()` against the real email API, (d) write `write_flagged()`, (e) wire all three together and run once end-to-end on real (but low-stakes) data before trusting it daily.